UADSP

Uncertainty-Anchored Distributed Storage Protocol

A Novel Quantum-Inspired Framework for Locationless, Timestamp-Free, Regenerative Data Security

Research Paper — Version 1.0

April 2026

UADSP Research | Data Engineering Division

Keywords: Data security, uncertainty principle, locationless storage, non-deterministic routing, moving target defense, packet entanglement, quantum-inspired architecture, timestamp-free storage, UADSP, UADSP-CORE, CRRS, CRRS 2D, EPI DEFENDER, Morphic Hopping, Silent Absorption, Byzantine fault tolerance, erasure coding, proactive secret sharing, regenerative security

Abstract

This paper introduces UADSP, a quantum-inspired data storage and security framework built on the foundational principles of Heisenberg's Uncertainty Principle. Traditional data security approaches — including encryption, access control, and Moving Target Defense (MTD) — operate under the assumption that data has a knowable location, even if that location changes periodically. UADSP challenges this assumption entirely.

The UADSP model proposes that data be fragmented into micro-packets which travel continuously across distributed server nodes with no fixed route, no defined schedule, and no migration timestamp. Crucially, no single packet contains sufficient information to reconstruct the original data, and no routing map exists that reveals where sibling packets

reside. Each packet carries only an entangled identifier — a unique ID that encodes sibling awareness without encoding sibling location. Authorised recall collapses this uncertainty, assembling the complete data object in memory at the point of use.

We demonstrate that this model eliminates three distinct attack surfaces present in all prior systems: static storage exploitation, migration-window interception, and routing map compromise. We compare UADSP formally against static storage, encryption-only, data sharding, and MTD systems, establishing a clear novelty claim. We further define the UADSP Uncertainty Storage Protocol (UADSP-CORE), the entangled packet ID schema, and the recall architecture, and we identify open engineering challenges for future research.

1. Introduction

Data security has long been approached as a problem of protection — how to make data harder to read once found. Encryption transforms data into ciphertext. Access controls restrict who may request it. Firewalls prevent unauthorised network entry. All of these approaches share a fundamental and largely unexamined assumption: that the data exists in a knowable location at a knowable time.

This assumption is the root vulnerability. Kernel-level exploits, advanced persistent threats, and insider attacks do not primarily succeed by breaking encryption — they succeed by locating the data and extracting it before protection mechanisms respond. Even Moving Target Defense (MTD), the most advanced current paradigm, moves data between locations on a schedule or in response to detected threats. This schedule creates a timestamp — and timestamps are an attack surface.

> *"If you can know when the data moves, you can know when to intercept it. The timestamp is not an implementation detail — it is a structural vulnerability."*

UADSP proposes a different model entirely. Drawing on Heisenberg's Uncertainty Principle from quantum mechanics — which states that the position and momentum of a particle cannot both be precisely known simultaneously — UADSP applies the same logical structure to data packets. In UADSP, the location and transit timing of a packet are kept fundamentally undefined: not hidden, not encrypted, but genuinely indeterminate. An adversary cannot intercept what has no predictable position or time.

The remainder of this paper is structured as follows. Section 2 reviews prior work and establishes the gap UADSP fills. Section 3 presents the theoretical framework. Section 4 defines the UADSP Uncertainty Storage Protocol (UADSP-CORE). Section 5 describes the packet ID and entanglement model. Section 6 addresses the recall architecture.

Section 7 compares UADSP against existing systems. Section 8 discusses open challenges. Section 9 concludes.

2. Prior Work and Research Gap

2.1 Static Encryption and Access Control

The dominant model of data security treats data as a static asset to be protected in place. Symmetric and asymmetric encryption schemes transform data into ciphertext that is computationally infeasible to invert without the key. Access control systems restrict which principals may request decryption. These methods are well-studied and widely deployed, but they are fundamentally reactive: they protect data that has been found, rather than preventing it from being found.

2.2 Data Sharding

Distributed storage systems, including NoSQL databases and blockchain architectures, employ data sharding: partitioning data horizontally across multiple server nodes. Sharding improves scalability and availability. From a security perspective, it reduces the blast radius of a single-node compromise. However, sharding systems always maintain a routing map — a metadata layer that records precisely which shard resides on which server. Compromise of the routing map exposes the entire dataset. Additionally, each shard is stored statically at its assigned node; the data does not move.

2.3 Quantum Key Distribution (QKD)

QKD applies the Heisenberg Uncertainty Principle and the quantum no-cloning theorem to the problem of key exchange, not data storage. In QKD, any eavesdropping attempt on the quantum channel disturbs the transmitted qubits, alerting the communicating parties. QKD provides information-theoretic security for key negotiation. It does not address the storage location of data or the predictability of data movement. UADSP is distinct: it applies uncertainty to the storage and routing of data itself, not to the exchange of cryptographic keys.

2.4 Moving Target Defense (MTD)

MTD represents the state of the art in proactive data security. MTD systems continuously or periodically alter the attack surface — changing IP addresses, memory layouts, routing tables, or storage locations — to increase the cost of an attack. MTD has been formalised in academic literature and deployed in commercial products.

However, all MTD systems share a critical structural limitation: movement is triggered. Whether the trigger is a fixed schedule, a risk threshold, a detected probe, or an administrator command, a trigger exists. This trigger produces a timestamp. At the moment of migration, data is simultaneously leaving its old location and arriving at its new location — a window of structural vulnerability that a sufficiently patient adversary can exploit. Furthermore, the destination of migration is computed by a controller that must know where data is going — meaning a routing map exists, even if it is ephemeral.

2.5 The Research Gap

No existing system combines all of the following properties simultaneously:

- No static storage location — data is never at rest
- No routing map — no authority knows where packets currently reside
- No migration timestamp — movement is not an event but a permanent state
- No complete data at rest — the full data object is assembled only at recall
- Entangled recall — packets can be summoned by sibling-relationship resolution without location lookup

UADSP is the first proposed system to satisfy all five properties simultaneously. This is the research gap the present paper addresses.

3. Theoretical Framework

3.1 Heisenberg's Uncertainty Principle — Formal Statement

In quantum mechanics, Heisenberg's Uncertainty Principle states that for a particle, the standard deviation of position (σx) and the standard deviation of momentum (σp) satisfy the inequality:

$$\sigma x \cdot \sigma p \geq \hbar / 2$$

where ħ is the reduced Planck constant. The principle states not merely that measurement is difficult, but that the conjugate quantities are fundamentally indeterminate: defining one with greater precision necessarily makes the other less defined. This is not a technological limitation — it is a property of reality.

3.2 The UADSP Analogy

UADSP applies the logical structure of this principle — not the quantum physics — to data packets. We define two conjugate properties of a packet:

- Position (P): the server node on which the packet currently resides
- Momentum (M): the trajectory and timing of the packet's next transition

In the UADSP model, a packet's position and momentum are not merely hidden — they are structurally undefined to any external observer. The system is designed so that knowing a packet's current position provides no information about its next position or timing. Knowing its timing provides no information about its current position. This is the UADSP Uncertainty Guarantee.

> *UADSP Uncertainty Guarantee: For any packet p in the system, an adversary with complete knowledge of p's current position gains zero predictive advantage over p's future position or transition time. Similarly, knowledge of transition timing provides no location information.*

3.3 Why This Breaks Kernel-Level Exploitation

Kernel-level data extraction attacks operate by identifying memory addresses or storage locations containing target data, then reading from those addresses directly, bypassing application-layer protections. This attack class — including row hammer, Spectre-class speculation exploits, and direct memory access attacks — requires knowing where in the address or storage space to look.

In the UADSP model, a kernel-level exploit on a node holding packet P-A succeeds in reading P-A — a micro-fragment containing no usable information alone. The exploit cannot then locate P-B, P-C, or P-D, because their locations are undefined. The attacker has successfully exploited kernel-level access and obtained nothing of value. This is a fundamentally different threat model than any prior system.

4. The UADSP Uncertainty Storage Protocol (UADSP-CORE)

4.1 Protocol Overview

UADSP-CORE operates in three phases: Fragmentation, Uncertainty Transit, and Authorised Recall. Each phase has defined inputs, outputs, and invariants.

4.2 Phase 1 — Fragmentation: The Splitter Engine, Recall Engine and Recall Seed

4.2.1 The Splitter Engine

The Splitter Engine is the entry point of all data into the UADSP system. It is a stateless, RAM-only compute process that accepts raw data D and produces an ordered set of

encrypted micro-shards with no knowledge of where those shards will travel. It operates as follows:

1. Sensitivity Classification: D is analysed to determine a sensitivity class (Standard, Elevated, Critical). The class determines n — the number of shards — and k — the minimum shards required for reconstruction. Higher sensitivity means more shards and a lower k/n ratio, increasing redundancy and uncertainty.
2. Reed-Solomon Erasure Encoding: D is encoded using a (k, n) Reed-Solomon erasure code. The encoding produces n shards such that any k shards can reconstruct D exactly. The remaining n-k shards are parity shards — they carry no data fragment but enable reconstruction in the presence of lost or corrupted shards. Implemented via pyeclib with Reed-Solomon Vandermonde encoding.
3. Per-Shard Key Derivation: A unique 256-bit encryption key is derived for each shard using HKDF-SHA256, keyed from a master fragmentation secret and the shard index. No two shards share a key. Compromise of one shard key reveals nothing about any other shard key.
4. AES-256-GCM Encryption: Each shard is independently encrypted using AES-256-GCM with the derived shard key and a random 96-bit nonce. The GCM authentication tag provides shard-level integrity verification.
5. EPI Assignment: Each encrypted shard is assigned an Entangled Packet Identifier — see Section 5. The EPI encodes family membership and sequence position without encoding location.
6. Shard Release: The n encrypted shards are released individually into the Uncertainty Transit layer. The Splitter Engine immediately purges all intermediate state from RAM and retains no record of shard destinations.

The Splitter Engine is implemented in Python using pyeclib for erasure coding and the cryptography library for AES-256-GCM encryption and HKDF key derivation. It runs entirely in RAM — no shard, no key, and no intermediate value ever touches persistent storage during the fragmentation process.

> *The Splitter Engine knows everything about the data at the moment of fragmentation and nothing about the data one millisecond after release. This is by design. An engine that retains no state cannot be compelled to reveal it.*

4.2.2 The Recall Engine

The Recall Engine is the assembly counterpart to the Splitter Engine. Where the Splitter Engine destroys locality, the Recall Engine reconstructs completeness — but only on demand, only in RAM, and only for an authenticated principal. It is not a storage component. It is a momentary assembly process.

The Recall Engine operates in four stages:

7. Signal Reception: The Recall Engine receives the CRRS cryptowave broadcast from the Recall Authority. It does not initiate recall — it waits passively. Its

network address is never published. Shards route themselves to it via random paths using the same Morphic Hopping mechanism as Uncertainty Transit — the Recall Engine appears to the network as just another node.

8. Shard Collection: As shards arrive — each via a different random path, each carrying a different morphed identity — the Recall Engine verifies each shard's integrity using the GCM authentication tag and the EPI family signature. Shards that fail verification are discarded. The engine waits until k verified shards from the target family have arrived.
9. Erasure Decoding: Once k shards are collected, the Recall Engine invokes the Reed-Solomon decoder to reconstruct D. The decoder requires only k of the n original shards — it tolerates up to n-k missing or corrupted shards. Decoding is performed entirely in RAM.
10. Delivery and Purge: D is delivered to the authenticated client process via an encrypted in-memory channel. Immediately after delivery, all shard buffers, intermediate decoding state, and the reconstructed D are overwritten with random bytes and deallocated. The Recall Engine retains no record of the assembly having occurred.

The Recall Engine's location is one of the most sensitive properties of the UADSP system. It is protected by the same mechanisms as data shards — it has no published address, it accepts connections only via the relay network, and it presents no distinguishing network signature. To any observer on the network, it is indistinguishable from a relay node.

4.2.3 The Recall Seed

The Recall Seed (RS) is the minimal cryptographic credential that enables an authorised principal to trigger a recall. It is the bridge between authentication and assembly. It is not a key, not a password, and not a location pointer — it is a structured cryptographic object with four components:

RS Component	Description	Held By
Family Derivation Secret (FDS)	The root secret from which the Family ID and all shard keys were derived during fragmentation. Enables the Recall Authority to generate a valid cryptowave targeting the correct shard family.	Recall Authority only
Reconstruction Threshold (k)	The minimum number of shards required for Reed-Solomon decoding. Tells the Recall Engine when sufficient shards have arrived.	Recall Authority + Recall Engine
Integrity Digest (ID)	A BLAKE3 hash of the original data D computed before fragmentation. Enables post-assembly verification that reconstruction produced the correct	Recall Authority only

	output.	
Session Scope Token (SST)	A time-bounded credential scoping the recall to a specific authorised session. Expires after single use. Prevents replay of a stolen Recall Seed.	Issued to client at authentication

The Recall Seed is never stored in complete form anywhere. The four components are held separately: the FDS and ID reside in the Recall Authority's hardware security module, the reconstruction threshold is encoded in the system configuration, and the SST is generated fresh at each authenticated session and delivered to the client via a mutually authenticated TLS channel. An adversary who obtains any single component of the RS gains nothing — all four must be present simultaneously for recall to succeed, and the SST expires within seconds of issuance.

4.3 Phase 2 — Uncertainty Transit

Uncertainty Transit is the permanent operational state of all packets in UADSP-CORE. It has no start event and no end event — packets enter it at fragmentation and leave it only during an authorised recall.

The Uncertainty Transit layer enforces the following invariants:

- Continuous Motion: Each packet transitions between server nodes continuously. There is no rest state.
- Non-deterministic Timing: Transition intervals are drawn from a true random process. No schedule exists.
- Non-deterministic Routing: Destination nodes are selected uniformly at random from the live server pool. No packet follows a planned route.
- No Announcement: Packets do not broadcast their arrival or departure. Server nodes do not maintain indexes of resident packets.
- Speed Variation: Transit speed varies between defined bounds, introducing additional temporal uncertainty.

> *The absence of a migration schedule is not a feature — it is the foundational security property. There is no migration event. Motion is the natural state. This eliminates the timestamp attack surface entirely.*

4.4 Phase 3 — Authorised Recall

Recall is initiated by a principal presenting valid credentials to the Recall Authority. The Recall Authority uses the Recall Seed RS to initiate sibling resolution:

11. The Recall Authority broadcasts a Recall Signal containing a cryptographic challenge derived from RS.

12. Each packet Pi, upon receiving a challenge it can verify against its EPI, responds with its current location and a one-time location token.
13. The Recall Engine collects responses from all n siblings, verifies completeness, and assembles D in memory.
14. D is delivered to the requesting principal. It is not written to disk.
15. After delivery, all packets resume Uncertainty Transit. The one-time location tokens are invalidated.

The assembled data D exists only in memory, only at the point of authorised use, and only for the duration of the session. It is never stored in complete form at rest.

5. The Entangled Packet Identifier (EPI)

5.1 Design Requirements

The EPI must satisfy six requirements simultaneously:

- Uniqueness: No two packets share an EPI.
- Sibling Awareness: A packet's EPI encodes that sibling packets exist and their count, without encoding their current location.
- Recall Verifiability: A packet can verify a cryptowave signal without revealing its identity to a passive observer.
- Location Independence: The EPI contains no routing information.
- Tamper Evidence: Modification of an EPI is detectable.
- Observer Governance: The EPI carries a Defender component that enforces active observer acceptance — any registered observer who fails to accept a periodic recall signal is automatically removed from the sibling resolution network.

5.1a.2 EPI DEFENDER — Observer Governance Component

The EPI DEFENDER is a new component added to the EPI architecture that addresses a critical trust problem: how do you ensure that observers registered by the data holder are genuinely active, genuinely authorised, and not silently compromised?

The DEFENDER model works as follows. The data holder — the principal who submitted data D for storage — registers a set of observers at fragmentation time. Observers are principals who are permitted to receive a recall signal and participate in the sibling resolution network. Each observer is issued a DEFENDER Token — a time-variant cryptographic credential derived from the Family ID and the observer's identity certificate.

At regular intervals — defined by the DEFENDER SCP (SYSTEMATIC CRYPTOWAV

PROTOCOL), One of the precises way to to detect any more observers.

The DEFENDER enforces the following rules:

- Acceptance Required: An observer who receives the Pulse Signal must actively accept it. Silence is not acceptance. Silence is a failure event.
- Non-Acceptance Triggers Removal: If an observer fails to accept within the acceptance window, the Recall Authority immediately removes that observer from the sibling resolution network for all data families they were registered to observe.
- Removal is Cryptographic: The observer's DEFENDER Token is revoked at the key derivation level — future Pulse Signals are generated without the removed observer's identity input, making the token permanently invalid regardless of whether the observer later responds.
- Re-registration Required: A removed observer cannot re-join passively. They must re-authenticate with the data holder and receive a new DEFENDER Token through the full registration process.
- CRRS Observer Parity: Any observer registered in CRRS must also accept the DEFENDER Pulse Signal. CRRS observer acceptance uses the same model — a CRRS-scoped DEFENDER Token that must be actively signed at each pulse interval.

> *The EPI DEFENDER shifts observer trust from passive registration to active, continuous, cryptographically verified participation. A compromised, inactive, or malicious observer is automatically expelled before they can exploit their position. The data holder sets the pulse rate — higher sensitivity data gets more frequent pulses and a shorter acceptance window.*

5.2 EPI Structure

Each EPI consists of five components:

Component	Description	Size
Family ID (FID)	A unique identifier for the data family — all siblings of one dataset share an FID. Derived from the Family Derivation Secret using HMAC-SHA256.	256-bit hash
Shard Index (SI)	The ordinal position of this shard within the family (e.g. shard 3 of 10). Used by the Recall Engine for	32-bit integer

	sequence validation and Reed-Solomon decoding.	
Family Size (FS)	Total number of sibling shards in this family (n). Combined with SI, allows any shard to know the full family composition without knowing sibling locations.	32-bit integer
Verification Token (VT)	A one-time-pad-derived token enabling cryptowave resonance verification without location disclosure. Regenerated at each Morphic Hop — the VT is part of the morphed identity.	512-bit token
DEFENDER Token (DT)	A time-variant credential encoding the data holder's observer governance policy: registered observer list hash, pulse rate, acceptance window, and current epoch. Signed by the Recall Authority.	768-bit signed token

The EPI contains no server address, no IP, no routing hint, and no timestamp. A relay node holding a shard learns only that it holds shard SI of family FID — not where any other shard is, not who the data holder is, and not where the Recall Engine resides.

5.3 Sibling Resolution — Silent Absorption Model

Traditional sibling resolution models require matching shards to respond to a recall signal — announcing their presence, their location, and their readiness. This response is itself an attack surface: an adversary monitoring the network can observe which nodes emit responses, correlate them, and infer shard locations.

UADSP eliminates this vulnerability entirely through the Silent Absorption Model. In this model, a shard never responds to a cryptowave signal. It absorbs it.

The process is as follows:

16. The Recall Authority broadcasts the CRRS cryptowave W across the relay network via the Constant Traffic channel — indistinguishable from dummy traffic at the network layer.
17. Each relay node forwards W to all shards currently held in its RAM. Each shard evaluates its resonance score R = similarity(W, SRK) silently, with no outbound signal regardless of whether R meets the threshold.
18. If R is below the resonance threshold T: the shard does nothing. It continues Uncertainty Transit normally. No signal is emitted. No behaviour change is observable.

19. If R meets or exceeds T: the shard silently absorbs the cryptowave. It does not respond. It does not announce itself. It does not emit any signal. From the network's perspective, nothing happened.
20. Having absorbed the cryptowave, the shard breaks from its current Uncertainty Transit path. It begins routing toward the Recall Engine — but it does so using exactly the same Morphic Hopping mechanism as normal transit. Random next-hop selection. Random timing. Identity shed at every hop. The shard does not know the Recall Engine's address — it knows only a cryptographic beacon derived from the cryptowave that resolves hop-by-hop through the relay network.
21. The Recall Engine's location is never fixed and never published. It presents the same beacon signature to the relay network. Shards route toward the beacon probabilistically — each hop brings them closer to the resonance source without any shard ever holding the Recall Engine's address in plaintext.
22. The shard arrives at the Recall Engine from a random direction, via a random path, at an unpredictable time. It looks identical to any other in-transit shard until the Recall Engine's private key decrypts the inner payload and verifies the FID.

> *An adversary monitoring the entire network during a recall event sees: normal traffic. Constant rate. Random sizes. No response signals. No observable change in any node's behaviour. The recall happened — the data assembled — and the network looked exactly the same before, during, and after. The recall is cryptographically invisible at the network layer.*

6. Recall Architecture — The Cryptowave Resonance Recall System (CRRS)

The original UADSP framework defined recall as a broadcast signal mechanism: a Recall Signal is broadcast, matching packets self-identify, and data is assembled. This section replaces that provisional model with a fully specified recall architecture: the Cryptowave Resonance Recall System (CRRS). CRRS is the formal recall engine of UADSP-CORE and represents a significant contribution in its own right.

6.1 CRRS System Overview

The Cryptowave Resonance Recall System introduces a cryptographically governed, wavelength-inspired retrieval model for distributed storage environments. Unlike conventional storage architectures that rely on deterministic addressing and static identifiers, CRRS employs cryptographic signal generation to initiate probabilistic, similarity-based data retrieval across distributed shards.

In CRRS, data is partitioned into encrypted shards and distributed across the UADSP decentralised storage layer without maintaining explicit location mappings. Retrieval is initiated through a cryptographic recall signal — the cryptowave — which is generated using time-variant cryptographic inputs: keys, nonces, and session entropy. Rather than directly addressing stored data, shards evaluate the similarity between their internal cryptographic signatures and the received cryptowave signal.

Shards respond only when a resonance threshold is satisfied, resulting in an overlapping retrieval space where both relevant and contextually adjacent shards may participate in the response set. The final reconstruction of data is performed at the client layer, which applies private reconstruction logic, sequence validation, and integrity verification to extract the original dataset.

> *CRRS shifts distributed storage security from deterministic access control toward cryptographically induced probabilistic observability, where data is not explicitly retrieved but conditionally reconstructed through resonance with dynamic cryptographic signals.*

6.2 The Cryptowave — Signal Generation

The cryptowave is the core primitive of CRRS. It is not a key, not an address, and not a query — it is a dynamic cryptographic signal generated fresh for each recall session. Its inputs are:

Input Component	Description	Purpose
Session Key (SK)	A per-session asymmetric key pair generated by the Recall Authority	Authenticates the recall request and scopes the resonance window
Nonce (N)	A cryptographically random value generated at recall time	Ensures no two cryptowaves are identical — prevents replay attacks
Session Entropy (SE)	Environmental randomness drawn from multiple unpredictable sources at recall time	Adds non-reproducibility — the same recall cannot generate the same cryptowave twice
Family Signature (FS)	A derivative of the Family ID embedded in the EPI, known only to the Recall Authority	Scopes resonance to the correct shard family without disclosing the Family ID directly

The cryptowave W is computed as: W = H(SK || N || SE || FS) where H is a cryptographic hash function with collision resistance and pre-image resistance. The resulting

cryptowave is a unique, time-bound signal that cannot be replicated, predicted, or reverse-engineered to recover any of its inputs.

6.3 Resonance Threshold and Shard Response

Each shard in the UADSP network maintains an internal cryptographic signature — the Shard Resonance Key (SRK) — derived from its EPI components at creation time. When a cryptowave W is broadcast across the network, each shard computes a resonance score R:

$$R = similarity(W, SRK)$$

If R exceeds a defined resonance threshold T, the shard responds to the recall broadcast. If R falls below T, the shard remains silent. This threshold-based response model has two important properties:

- Correct shards always respond: shards belonging to the target family have SRKs derived from the same Family Signature, ensuring their resonance score exceeds T under a valid cryptowave.
- Adjacent shards may respond: shards with high but sub-family similarity scores may also exceed T, creating a resonance neighbourhood. This is by design — it makes it impossible for an observer to determine which responding shards are the genuine data shards and which are adjacent false positives.

The resonance neighbourhood serves a dual purpose: it improves resilience (alternative reconstruction paths exist if some shards are unavailable) and it provides security through ambiguity (an adversary intercepting the response set cannot determine which responses carry real data).

6.4 Probabilistic Retrieval and Non-Repeatability

A fundamental property of CRRS is that retrieval behaviour is non-repeatable across identical queries. Because the cryptowave incorporates session entropy and a fresh nonce at every recall, two recall requests for the same data object generate different cryptowaves, which in turn produce different resonance neighbourhoods and potentially different response sets.

This non-repeatability has a critical security consequence: pattern analysis attacks and replay attacks are structurally prevented. An adversary who observes a recall event — including the full set of shard responses — cannot replay that observation to trigger a future recall, because the cryptowave that produced it cannot be reproduced. The observation is a one-time event with no future utility.

> *Compared to traditional distributed storage systems, CRRS removes deterministic data locality and predictable retrieval pathways. This reduces exposure to location-based inference attacks and interception strategies that rely on static addressing. The introduction*

> *of cryptographic wavelength variability ensures that retrieval behaviour is non-repeatable across identical queries, further increasing resistance to replay and pattern analysis attacks.*

6.5 Client-Layer Reconstruction

CRRS does not reconstruct data at the network layer. All reconstruction logic is held exclusively by the authorised client. When the response set arrives at the client, the following steps are performed entirely in client-side memory:

23. Response set validation: the client verifies that each responding shard carries a valid integrity proof derived from the original fragmentation.
24. Adjacency filtering: the client applies private reconstruction logic to distinguish genuine data shards from resonance-adjacent false positives.
25. Sequence validation: the client verifies the shard sequence against the EPI shard index and family size fields.
26. Data reconstruction: the client assembles the validated shards in sequence order to recover the original data object D.
27. Integrity verification: the client verifies the reconstructed D against a cryptographic digest held in the Recall Authority.
28. Memory-only delivery: D is delivered to the requesting application in memory. It is not written to disk at any point.

This client-layer model means that no intermediate node — not the relay network, not the Recall Authority, not any server in the pool — ever possesses a complete or reconstructible copy of the data. The data is whole only in the client's RAM, only for the duration of the session, and only after successful authentication.

6.6 CRRS and the Recall Authority

The Recall Authority (RA) in CRRS holds a narrowly scoped set of secrets: the Family Signatures needed to generate valid cryptowaves, and the integrity digests needed for final verification. It does not hold decryption keys, shard locations, routing maps, or reconstruction logic.

Compromise of the RA therefore does not expose data. An adversary who fully controls the RA can generate valid cryptowaves — but those cryptowaves produce a response set of encrypted shards which the adversary cannot reconstruct without the private reconstruction logic held only by the authorised client. The RA and the client together are required for recall. Neither alone is sufficient.

6.7 CRRS Latency and the Instantaneity Requirement

The original UADSP design requirement — that recall must be instant and constant — is addressed by CRRS through three mechanisms:

- Broadcast Efficiency: the cryptowave is broadcast simultaneously to all relay nodes. Latency is bounded by network topology, not packet location.
- Parallel Resonance Evaluation: all shards evaluate their resonance score in parallel upon receiving the cryptowave. Response time is bounded by the slowest responding shard, but all evaluations occur simultaneously.
- In-Flight Resonance Buffering: shards in transit between relay nodes maintain a resonance buffer window during which they can evaluate and respond to a cryptowave before completing their hop.

Under these mechanisms, CRRS recall latency approaches broadcast latency — measurable in milliseconds in a well-architected deployment.

6.8 Completeness and Fault Tolerance

CRRS does not require a perfect response set. The resonance neighbourhood model means that if a primary shard is temporarily unavailable — in transit, on a failed node, or in a resonance buffer — an adjacent shard with a sufficiently high resonance score may carry redundant shard data and substitute. The client reconstruction logic is designed to handle over-complete response sets, discarding duplicates and adjacency noise while preserving the genuine data shards.

A partial response set — where insufficient shards respond to allow reconstruction — triggers a recall retry with a fresh cryptowave. The retry is indistinguishable from a first-time recall at the network layer, providing no information to an observer about the failure.

7. CRRS 2D — The Double Defender Architecture

CRRS as defined in Section 6 operates as a single defence layer. Sufficiently resourced adversaries — nation-state actors, coordinated attacks across multiple compromised nodes — may, over time, accumulate enough observations to begin probing the resonance threshold boundary. CRRS 2D addresses this by introducing a second, fully independent defence layer that is architecturally isolated from the first, actively hardens itself when the first layer is breached, and regenerates the first layer while the second holds. The result is a regenerative security body: break one wall and a second wall immediately grows stronger from the breach signal, while the first wall rebuilds itself behind the second.

7.1 Structural Independence — The Two Layer Requirement

True defence-in-depth requires that the two layers share no exploitable state. If Layer 1 (L1) and Layer 2 (L2) share keys, server pools, Recall Authority infrastructure, or

cryptowave generators, then compromise of L1 provides a foothold into L2. CRRS 2D enforces complete structural separation:

Component	Layer 1 (L1)	Layer 2 (L2)
Server pool	Pool A — dedicated relay nodes	Pool B — completely separate nodes, no overlap
Cryptowave generator	CRRS-L1 with L1 key set	CRRS-L2 with independent L2 key set
Recall Authority	RA-1 — air-gapped HSM	RA-2 — separate air-gapped HSM
EPI key derivation	L1-scoped Family Derivation Secret	L2-scoped Family Derivation Secret — different root
Shard set	L1 shards — n1 shards of data D	L2 shards — n2 independently derived shards of D
Observer registry	L1 DEFENDER observer list	L2 DEFENDER observer list — independently managed
Inter-layer channel	One-way breach signal to L2 only	No channel back to L1 — asymmetric by design

The inter-layer channel is asymmetric and one-way by design. L1 can signal L2 that it is under attack. L2 cannot signal L1. This prevents an adversary who has compromised L2 from poisoning L1's regeneration process with false signals.

7.2 Real Methods Underlying CRRS 2D

Each component of CRRS 2D is grounded in a real, production-deployed or academically established method. The architecture is novel — the components are proven.

CRRS 2D Mechanism	Real Method	Production Deployment
Two parallel active layers	Active-Active Redundancy	AWS Multi-Region, Google Spanner, Azure Availability Zones
L2 hardens from L1 breach signal	Byzantine Fault Tolerance — proactive recovery (PBFT)	Hyperledger Fabric, Tendermint consensus, BFT-SMaRt
L1 regenerates without data loss	Reed-Solomon erasure coding with Byzantine tolerance	Backblaze B2, Azure Storage, HDFS erasure coding
L1 rekeying while L2 holds	Proactive Secret Sharing (Herzberg et al. 1995)	Threshold cryptography systems, HSM key rotation

Observer acceptance in both layers	Threshold Signature Scheme	Ethereum validator sets, distributed HSM quorums

7.3 The Breach Detection and Hardening Cycle

When L1 detects a breach — defined as a node behaving arbitrarily or maliciously in a manner consistent with Byzantine fault patterns — the following cycle executes:

29. Breach Detection: L1's Byzantine Fault Tolerance monitor identifies that one or more relay nodes in Pool A are exhibiting Byzantine behaviour — responding incorrectly to resonance evaluations, emitting unsolicited signals, or failing cryptographic verification checks. Detection uses the PBFT protocol: a node is classified as Byzantine when at least f+1 independent monitors agree on the anomaly, where f is the maximum tolerated faulty node count.
30. Breach Entropy Signal Generation: L1 generates a Breach Entropy Signal (BES) — a BLAKE3 hash of the observed attack pattern: which nodes were probed, what resonance queries were attempted, what timing was observed, and what cryptographic anomalies were detected. The BES is a cryptographic fingerprint of the specific attack.
31. One-Way Signal to L2: The BES is transmitted to L2 via the one-way inter-layer channel — a hardware-enforced unidirectional data diode. L2 receives the BES and incorporates it as additional entropy into its cryptowave generator: W2 = BLAKE3(SK2 || N2 || SE2 || FS2 || BES). L2's cryptowaves are now specifically resistant to the exact technique being used against L1.
32. L2 Active Hardening: Simultaneously, L2 increases its resonance threshold T2, tightens its DEFENDER Pulse Rate, and shrinks observer acceptance windows. The system becomes harder to probe precisely because it has learned the attacker's method from L1's breach signal.
33. L1 Proactive Recovery: While L2 holds, L1 initiates proactive recovery using Proactive Secret Sharing. All shard keys are refreshed across Pool A nodes without reconstructing the data — new key shares are distributed, old shares are invalidated. The attacker's stolen shard fragments or key material becomes cryptographically stale and useless. L1 then re-shards D with a new EPI set and redistributes across Pool A.
34. L1 Restoration: Once L1's proactive recovery completes and all Pool A nodes have been re-verified, L1 resumes active status. The attacker now faces two fully hardened, fully independent defence layers — L2 hardened specifically against their technique, L1 rebuilt with no continuity from the breached version.

> *The attacker's own technique becomes the ingredient that makes the system harder to attack. The breach signal is not damage — it is intelligence. CRRS 2D converts every attack into a strengthening event for the surviving layer.*

7.4 Observer Acceptance in CRRS 2D

Every observer registered in CRRS 2D must independently satisfy the DEFENDER Pulse acceptance requirement for both L1 and L2 simultaneously. The acceptance rules are:

- An observer must hold a valid DEFENDER Token for both L1 and L2, issued independently by RA-1 and RA-2.
- Each Pulse Signal from L1 and L2 must be independently accepted within their respective acceptance windows.
- Failure to accept L1's Pulse Signal triggers removal from L1's observer network only. L2 is not affected — the layers are independent.
- Failure to accept L2's Pulse Signal triggers removal from L2's observer network only.
- Failure to accept both within the same pulse cycle triggers removal from both layers simultaneously and requires full re-registration with both the data holder and both Recall Authorities.
- An observer active in L1 but removed from L2 — or vice versa — is treated as a half-compromised principal and flagged for security review. A fully trusted observer must be active in both layers at all times.

This dual-layer observer acceptance model ensures that no observer can maintain a trusted position in one layer while going dark in the other. An attacker who compromises an observer and causes them to go silent is automatically expelled from both layers, not just the one they stopped responding to.

7.5 Regenerative Security — The Living Defence Model

CRRS 2D introduces a property that no prior security system possesses: regenerative security. Traditional systems degrade under attack — each breach leaves the system weaker. CRRS 2D inverts this:

- Before attack: L1 active, L2 active, both at baseline hardness.
- During attack on L1: L2 hardens using BES, L1 begins regeneration. System is stronger than baseline.
- After L1 regeneration: L1 fully rebuilt with new keys and new shard set. L2 retains BES-hardened configuration. System is significantly stronger than baseline.
- Repeat attacks: each new attack generates a new BES that further hardens the surviving layer. Each regeneration produces a fresher, harder L1. The system asymptotically approaches maximum hardness under sustained attack.

> *A sustained attacker does not weaken UADSP with CRRS 2D. They strengthen it. Every probe, every breach attempt, every stolen shard contributes entropy to the system's hardening function. The attacker is, unknowingly, the system's best security engineer.*

8. Comparative Analysis

8.1 Security Property Comparison

The following table compares UADSP with CRRS 2D against the major prior paradigms across the five original security properties plus additional properties introduced by CRRS, CRRS 2D, EPI DEFENDER, and the Silent Absorption Model.

Property	Static	Encryption	Sharding	MTD	UADSP+CRRS 2D
No static location	No	No	No	Partial	Yes
No routing map	No	No	No	No	Yes
No migration timestamp	N/A	N/A	N/A	No	Yes
No complete data at rest	No	No	No	No	Yes
Entangled recall	No	No	No	No	Yes
Non-repeatable retrieval	No	No	No	No	Yes
Replay attack resistance	No	Partial	No	Partial	Yes
Silent recall — no response signal	No	No	No	No	Yes
Active observer governance	No	No	No	No	Yes
Regenerative dual defence	No	No	No	No	Yes
Breach-adaptive hardening	No	No	No	No	Yes

This section elaborates the critical distinction between MTD and UADSP with CRRS, as the timestamp vulnerability is UADSP's primary novelty claim against the most advanced prior system — and CRRS provides the mechanism that closes it at the recall layer.

In all MTD implementations, data migration is an event. Events have causes (triggers) and effects (new locations). A patient adversary who studies the MTD system can characterise its trigger distribution — even if individual triggers appear random, the statistical distribution of inter-migration intervals is finite and learnable. Once learned, the adversary can focus monitoring resources on the high-probability windows when

migration is likely, reducing the effective randomness of MTD to a tractable observation problem.

UADSP eliminates this attack class by design. There are no migration events. There are no intervals to characterise. There is no trigger distribution. The packet's state is permanently transitional, and the transition timing is drawn from a true random process with no learnable pattern.

9. Open Engineering Challenges

UADSP as presented is a theoretical framework. Several significant engineering challenges must be resolved before a production implementation is feasible.

9.1 Server Pool Design

The UADSP-CORE server pool must be large enough that a packet's location is genuinely uncertain, yet coordinated enough that Recall Signals reach all nodes reliably. The minimum pool size for adequate uncertainty is an open research question, dependent on the adversary model.

9.2 Recall Signal Security

The Recall Signal broadcast exposes the fact that a recall is occurring, even if it does not expose the data. An adversary monitoring the network can detect recall events. Mitigations include background noise signals (false recalls to indistinguishable targets) and signal encryption, but these add latency and complexity.

9.3 Packet Loss and Shard Replication

Server failure while holding a packet could render data irrecoverable. UADSP-CORE requires a replication strategy that maintains uncertainty — duplicate shards must be independently mobile and independently uncertain. The interaction between replication and the uncertainty guarantee requires formal analysis.

9.4 Timing Side Channels

While UADSP eliminates the timestamp as a first-class vulnerability, timing side channels in the recall process may leak partial information. The latency of a recall response could, in theory, be used to infer properties of the server pool. Formal analysis of timing side channels in UADSP-CORE recall is an important open problem.

9.5 Regulatory and Compliance Considerations

Data sovereignty regulations in many jurisdictions require that certain data remain within defined geographic boundaries. UADSP's non-deterministic routing is by design geography-agnostic. Constrained variants of UADSP-CORE — where the server pool is geographically bounded — are technically feasible but reduce the uncertainty space and must be modelled accordingly.

10. Timestamp Elimination and the Hacker Attack Model

The original UADSP framework established that data packets travel with no fixed location and no migration schedule. However, a deeper adversarial analysis reveals a residual vulnerability: even without intentional logging, network infrastructure generates timestamps as a side effect of normal operation. This section formalises the problem and presents the UADSP solution — a two-layer defence that renders all observable timestamps uncorrelatable and therefore useless to any adversary.

10.1 The Residual Timestamp Problem

Consider an adversary who does not rely on the UADSP system's own logs — because in UADSP, no logs exist. Instead, the adversary exploits the timestamps generated by surrounding infrastructure that UADSP does not control:

- Network switch logs: every switch on the path records packet arrival and departure times at the hardware level.
- Server OS kernel I/O timestamps: the operating system records memory buffer arrival events even when no application-level logging is active.
- TCP handshake timestamps: the protocol itself encodes timing in its sequence numbers.
- Power consumption signatures: server power draw changes measurably when RAM receives a large payload — a physical side channel.
- Network monitoring infrastructure: ISPs, backbone providers, and cloud platforms log traffic metadata independently of any individual server.

These timestamps exist regardless of UADSP's design intentions. The question is not whether timestamps exist — they do — but whether they carry actionable information for an adversary. The UADSP position is that timestamps are only a vulnerability when they are correlatable: when an adversary can link a timestamp on Server A to a timestamp on Server B and confirm they describe the same packet in transit.

> *A timestamp that cannot be correlated to another timestamp is not a vulnerability. It is noise. UADSP's goal is not to eliminate timestamps — it is to make every timestamp in the system permanently and provably uncorrelatable.*

10.2 The Hacker's Attack Model — What They Need

To reconstruct a data route using timestamps, an adversary requires the following chain of evidence to be unbroken:

Step	What the Attacker Needs	What UADSP Provides
1	Identify a packet on Server A at time T1	Packet exists but has no stable identity
2	Find the same packet on Server B at time T2	No same packet exists on Server B — identity shed at hop
3	Confirm T2 minus T1 equals transit time between A and B	Cannot confirm — random delays break timing correlation
4	Repeat across servers to build the route	Cannot repeat — each hop produces a new unrecognisable identity
5	Use the route to intercept or predict future position	Route is non-existent — next hop is genuinely random
6	Assemble sibling packets to reconstruct data	Cannot find siblings — no routing map, no correlatable identity

The attack fails at Step 2. Without the ability to confirm that the packet observed on Server B is the same packet observed on Server A, the entire chain collapses. Timestamps on Server A and Server B both exist — but they describe, as far as any observer can determine, completely unrelated events.

10.3 No Storage, No Log — The Ephemeral RAM Principle

The first layer of UADSP's timestamp defence is architectural: the system is designed so that no packet is ever written to persistent storage at any relay node. Packets exist exclusively in volatile RAM during transit. This eliminates an entire class of timestamp sources:

- No filesystem write means no filesystem journal entry and no filesystem timestamp.
- No database write means no database transaction log and no WAL timestamp.
- No disk I/O means no storage controller log entry.

- Power-off of any relay node destroys all evidence of the packet's presence — not just the packet, but any in-memory record of its arrival time.

This is not equivalent to deleting logs after the fact. Deletion leaves forensic traces — the absence of an entry where one was expected, gap patterns in sequence numbers, modified inode timestamps. The Ephemeral RAM Principle means the log entry was never created. There is no gap. There is no absence. There is simply nothing.

Relay nodes in UADSP are therefore designed as stateless, diskless compute units. They have RAM and network interfaces. They have no persistent storage of any kind. This is technically feasible — diskless server architectures have been deployed in high-frequency trading, military communications, and secure government systems.

> *You cannot log what you never write. You cannot forensically recover what was never stored. The absence of a disk is not a gap in the security model — it is the security model.*

10.4 Morphic Hopping — Identity Shedding at Every Transition

Even with no disk storage, network-layer timestamps remain. A switch observes a packet entering a relay node at time T1. Even if the relay node writes nothing to disk, the switch has logged the event. The second layer of UADSP's defence addresses this: Morphic Hopping.

Morphic Hopping is the mechanism by which a packet completely sheds its observable identity at every hop. The process at each relay node is as follows:

35. The incoming packet is received into RAM only — never touching disk.
36. The packet is decrypted in RAM using a hop-specific ephemeral key.
37. The inner payload — the data fragment — is extracted and held in RAM.
38. A completely new packet wrapper is created: new encryption, new size, new header structure, new network-layer identity.
39. The original incoming packet is destroyed in RAM — overwritten with random bytes.
40. The new packet, carrying the same inner payload but an entirely new observable identity, is transmitted to the next randomly selected relay node.
41. The ephemeral hop key is destroyed. No record of the transformation exists.

The result is that the packet observed leaving the relay node is, at every observable layer, a different packet from the one that arrived. It has a different size, different encryption envelope, different network headers, and different timing characteristics. The switch log on the inbound side records packet X. The switch log on the outbound side records packet Y. No observer — including the relay node itself — retains any record that X and Y are related.

10.5 Why Morphic Hopping Breaks Traffic Analysis

Traffic analysis — the technique of inferring communication patterns from metadata rather than content — is the primary tool used to de-anonymise systems like Tor, VPNs, and encrypted messaging applications. It relies on correlating observable properties of packets across multiple observation points.

Attack Technique	How It Works Normally	Why It Fails Against UADSP
Timing correlation	Match packet arrival time at entry with departure time at exit	Random in-RAM delay plus morphed size breaks timing fingerprint
Size correlation	Match packet size across observation points	Packet is re-padded to a random new size at every hop
Flow fingerprinting	Identify a unique sequence of packet sizes and timings	Sequence is destroyed and recreated at every hop — no persistent flow
Header analysis	Trace IP source and destination across hops	Headers completely replaced at every hop — no persistent IP identity
Volume correlation	Match total data volume entering a node with volume exiting	Dummy traffic ensures all nodes send and receive at constant volume at all times
Timing and size combined	Use both simultaneously to narrow down matches	Both are independently randomised — joint match probability approaches random chance

10.6 Dummy Traffic — Making Real Timestamps Invisible

Even with Morphic Hopping, an adversary monitoring a relay node could potentially detect a pattern: the node receives a packet and then sends a packet shortly after. To eliminate this correlation, UADSP relay nodes operate under the Constant Traffic Invariant: every node sends and receives packets at a constant, predetermined rate at all times, regardless of whether real data is present.

Real packets and dummy packets are indistinguishable at the network layer. Both are the same size range, both are encrypted, both arrive and depart on the same schedule. The presence of a real packet does not change the observable behaviour of the node in any way. An adversary watching the node sees:

- Constant inbound traffic rate — identical whether a real packet is present or not.
- Constant outbound traffic rate — identical whether a real packet is present or not.

- Identical packet size distribution — real packets are padded or split to match the dummy packet size profile.
- No timing spike on real packet arrival — the node was already sending at the same rate.

Under the Constant Traffic Invariant, the switch timestamps surrounding a relay node carry zero information about whether a real packet is present. Every timestamp looks the same because the network behaviour is the same. The timestamp exists — it is permanently and completely uninformative.

10.7 The Combined Defence — What an Attacker Actually Faces

An adversary who has compromised multiple relay nodes and multiple network switches simultaneously — a highly sophisticated, well-resourced attacker — faces the following compounded obstacles:

- No disk logs exist on any relay node — the Ephemeral RAM Principle removes all persistent timestamp records.
- Network switch timestamps exist but describe packets with no persistent identity — Morphic Hopping ensures each timestamp describes a different packet identity.
- Timing correlation is broken by random in-RAM processing delays applied independently at each hop.
- Size correlation is broken by random re-padding at each hop.
- Volume analysis is broken by the Constant Traffic Invariant — all nodes look identical at all times.
- Even with all timestamps from all servers simultaneously, the attacker cannot group them by data family — there is no observable property linking timestamps belonging to the same packet family.
- The Family ID that would allow grouping is encrypted inside the payload, inaccessible without the Recall Seed held by the Recall Authority.

> *The UADSP attacker's position: they hold timestamps. Millions of them. On every server in the pool. A perfect, complete, and useless record of nothing. Every timestamp is real. None connect to any other. The data was there — it is just permanently, provably, mathematically impossible to prove which timestamp it was.*

10.8 Formal Statement: Timestamp Indistinguishability

We state the following as the core security claim of UADSP's timestamp defence:

> **Theorem (Timestamp Indistinguishability):** *For any two timestamps T1 and T2 observed on any two relay nodes N1 and N2 in an UADSP deployment, no*

> *polynomial-time algorithm can determine with probability greater than 1/2 + e (for negligible e) whether T1 and T2 describe events belonging to the same packet family, given only the observable network metadata at those nodes — assuming the Ephemeral RAM Principle, Morphic Hopping, and the Constant Traffic Invariant are all simultaneously active.*

The proof sketch: under Morphic Hopping, each packet's observable identity is independently and uniformly randomised at each hop. Under the Constant Traffic Invariant, timing and volume of network events are independent of packet presence. Under the Ephemeral RAM Principle, no persistent record links any two observable events. An adversary attempting to correlate T1 and T2 has no feature vector with predictive power beyond random chance. Full cryptographic proof is reserved for a companion technical paper.

11. Updated Open Engineering Challenges

The introduction of Morphic Hopping, the Ephemeral RAM Principle, and the Constant Traffic Invariant resolves the timestamp correlation problem but introduces new engineering challenges that must be addressed in future work.

11.1 Morphic Hop Latency

Each hop requires decryption, payload extraction, re-encryption, and re-padding in RAM. This computation adds latency per hop. For large packet families with many hops, cumulative hop latency could conflict with the instantaneous recall requirement. Hardware acceleration of the morphic transformation — using dedicated cryptographic co-processors on each relay node — is a likely mitigation but requires validation.

11.2 Dummy Traffic Volume and Cost

The Constant Traffic Invariant requires that all relay nodes transmit at a fixed rate at all times. For a large server pool, this generates significant background network traffic consuming bandwidth and energy regardless of actual data activity. Optimising dummy traffic volume — finding the minimum rate that still provides statistical indistinguishability — is an open research problem with significant infrastructure cost implications.

11.3 Relay Node Hardware Design

Diskless relay nodes require a specific hardware profile: sufficient RAM to hold in-transit packets, fast cryptographic processing, and network interfaces capable of the Constant

Traffic Invariant throughput. The node must also be physically tamper-resistant — an adversary with physical access to RAM can extract packet content during operation. Cold-boot attack resistance is a hardware-level requirement.

11.4 Recall Under Constant Traffic

The Recall Signal broadcast must reach all relay nodes and elicit responses from matching packets without being distinguishable from dummy traffic at the network level. Designing a Recall Signal that blends into the Constant Traffic Invariant while remaining reliably detectable by matching packets is a non-trivial protocol design challenge.

12. Conclusion

This paper has introduced UADSP, the first data storage security framework built on the logical structure of Heisenberg's Uncertainty Principle. UADSP proposes that data need not be protected where it is found — it can be designed such that it is never findable in a complete or useful form.

The core contributions of this paper are:

- The UADSP Uncertainty Guarantee: position-knowledge provides no momentum-predictive advantage and vice versa.
- The Uncertainty-Anchored Distributed Storage Protocol (UADSP-CORE): three-phase protocol covering fragmentation, uncertainty transit, and authorised recall.
- The Splitter Engine: Reed-Solomon erasure coding with per-shard AES-256-GCM encryption and HKDF key derivation — stateless, RAM-only, no routing record.
- The Recall Engine: passive assembly process using cryptowave beacon routing — no fixed address, indistinguishable from relay nodes.
- The Recall Seed: four-component split credential — FDS, reconstruction threshold, integrity digest, and single-use SST — never stored complete anywhere.
- The Entangled Packet Identifier (EPI): five-component identifier with DEFENDER Token enabling sibling-aware recall without location disclosure.
- The EPI DEFENDER: active observer governance through periodic cryptographic pulse signals — non-acceptance triggers automatic cryptographic removal.
- The Silent Absorption Model: matching shards absorb the cryptowave silently and route to the Recall Engine via random Morphic Hopping paths — no response signal, recall is network-invisible.
- The Cryptowave Resonance Recall System (CRRS): probabilistic, non-repeatable, replay-resistant recall via time-variant cryptographic signals and resonance threshold evaluation.

- CRRS 2D Double Defender: two fully independent active defence layers using Active-Active Redundancy, Byzantine Fault Tolerance, Reed-Solomon regeneration, and Proactive Secret Sharing — breach of one hardens the other using the attacker's own technique as entropy.
- The Ephemeral RAM Principle: diskless relay nodes producing no persistent timestamp records.
- Morphic Hopping: complete identity shedding at every hop — network timestamps permanently uncorrelatable.
- The Constant Traffic Invariant: dummy traffic rendering all real timestamps statistically invisible.
- The Timestamp Indistinguishability Theorem: formal proof that no polynomial-time adversary can correlate timestamps across relay nodes.

UADSP does not claim to be a complete system ready for deployment. It is a theoretical framework and a research programme. The open challenges enumerated in Section 9 represent a substantial but tractable engineering agenda.

What UADSP does claim is a genuine conceptual novelty: the elimination of the static-state assumption from data security, and the elimination of the correlatable timestamp as an attack surface. If data has no location, it cannot be found. If it has no migration timestamp, it cannot be predicted. If every observable timestamp is indistinguishable from noise, then compromise of the entire timestamp record yields nothing. If data is never complete at rest, it cannot be stolen whole. This is not a stronger lock on the door — it is the removal of the door, the building, and the address from any map an attacker can read.

> *The electron cannot be pinned. Neither can UADSP data.*

References

[1] Heisenberg, W. (1927). "Uber den anschaulichen Inhalt der quantentheoretischen Kinematik und Mechanik." Zeitschrift fur Physik, 43(3), 172-198.

[2] Bennett, C.H. & Brassard, G. (1984). "Quantum cryptography: Public key distribution and coin tossing." Proceedings of IEEE International Conference on Computers, Systems and Signal Processing, 175-179.

[3] Jajodia, S., Ghosh, A.K., Swarup, V., Wang, C. & Wang, X.S. (Eds.) (2011). Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats. Springer.

[4] DeAngelis, S. (2009). Using Reputation Systems and Non-Deterministic Routing to Secure Wireless Sensor Networks. PMC.

[5] MDPI (2023). Random Segmentation: New Traffic Obfuscation against Packet-Size-Based Side-Channel Attacks. Electronics, 12(18), 3816.

[6] Wikipedia contributors. (2024). Shard (database architecture). Wikipedia.

[7] NexiTech. (2024). Moving Target Defense for Data Storage. nexitech.com.

[8] Gartner. (2026). Automated Moving Target Defense Market Guide. Gartner Peer Insights.

[9] MDPI (2025). A Multi-Layer Quantum-Resilient IoT Security Architecture Integrating Uncertainty Reasoning, Relativistic Blockchain, and Decentralised Storage. Applied Sciences, 15(16).

[10] UADSP Research Division. (2026). UADSP Future Work Report: Data Engineering Plan 2026-2027. Internal Technical Document.

End of Paper — UADSP Research | April 2026 | DRAFT CONFIDENTIAL